SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05
NAME: Mulesh Raj L
REG NO: 192571036

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

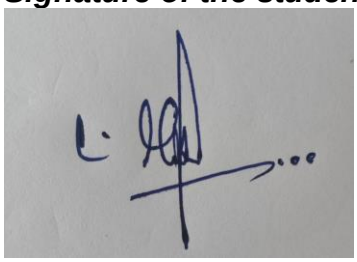
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and	BL4 – Analyze	**5**

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I M u k e s h R a j L (1 9 2 5 7 1 0 3 6) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context

Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1)

File Organization for E-Commerce Product Records

Recommended Strategy: Sorted File Organization with B+ Tree Index

1. File Organization

Store the records sorted by ProductID.

PRODUCT

ProductID
ProductName
Category
Price
Stock
SupplierID

Create a **B+ Tree primary index** on ProductID.

2. Why B+ Tree?

- Supports fast search using ProductID.
- Suitable for very large datasets.
- Supports both point searches and range searches.
- B+ Tree remains balanced after insertions and deletions.
- Leaf nodes are linked, making sequential and range access efficient.

3. Cost Analysis

Assume:

Number of records = 10,000,000
Average record size = 200 bytes
Block size = 4 KB = 4096 bytes

Records per block:

4096 / 200 ≈ 20 records

Approximate number of data blocks:

10,000,000 / 20
= 500,000 blocks
Search Cost

Sequential file:

Worst case ≈ 500,000 block accesses

B+ Tree indexed file:

For a large B+ Tree, only a small number of index levels need to be traversed.

Search ≈ 3–4 I/O operations

Therefore, the B+ Tree provides a huge improvement over scanning the complete file.

4. Comparison

Method	Search	Insertion	Range Query	Storage
Heap File	Slow	Fast	Slow	Low
Sorted File	Binary search	Costly	Fast	Medium
Sorted + B+ Tree	**Very Fast**	**Efficient**	**Very Fast**	Medium

Conclusion

For **10 million products**, the recommended design is a **sorted file organized by ProductID with a B+ Tree index**. It provides fast retrieval, efficient range queries, and good scalability, making it suitable for a large e-commerce database.

2)

Indexing Strategy for Banking Transactions

Recommended Indexing Strategy

Use a **B+ Tree composite index** on:

(AccountNo, TransactionDate)

1. Transaction Table

TRANSACTION

TransactionID
AccountNo
TransactionDate
Amount
TransactionType

2. Composite B+ Tree Index

INDEX ON (AccountNo, TransactionDate)

This index is suitable because the system needs:

- Fast retrieval for a particular AccountNo.
- Efficient retrieval of transactions within a **date range**.
- B+ Tree supports both equality and range searches.

Example Query

```
SELECT *
FROM TRANSACTION
WHERE AccountNo = 10025
AND TransactionDate BETWEEN '2026-08-01' AND '2026-08-15';
```

The index first locates the required account and then efficiently searches the specified date range.

3. Additional Index

If the system frequently searches transactions **only by date**, create a separate B+ Tree index:

INDEX ON (TransactionDate)

Comparison

Requirement	Suitable Index
Search by AccountNo	B+ Tree
AccountNo + Date Range	Composite B+ Tree
Date-only range search	B+ Tree on Date
Exact transaction lookup	Primary/unique index on TransactionID

Conclusion

The best strategy is a **composite B+ Tree index on** (AccountNo, TransactionDate). It provides fast account-based retrieval and efficient date-range queries while reducing disk I/O.

3)

Multi-Level Indexing for Healthcare Records

Recommended Solution

Use **two separate B+ Tree indexes** because the records need to be searched using two different attributes.

1. Patient Records

PATIENT

PatientID
PatientName
DoctorName
Age
Diagnosis

2. Primary Index on PatientID

Create a **multi-level B+ Tree index** on PatientID.

Level 1: Root Index
↓
Level 2: Intermediate Index
↓
Level 3: Leaf Index
↓
Patient Records

This provides fast retrieval when the PatientID is known.

3. Secondary Index on DoctorName

Create a **secondary B+ Tree index** on DoctorName.

DoctorName
↓
B+ Tree Index
↓
Patient Record Locations

This allows the system to quickly find all patients associated with a particular doctor.

Example Queries

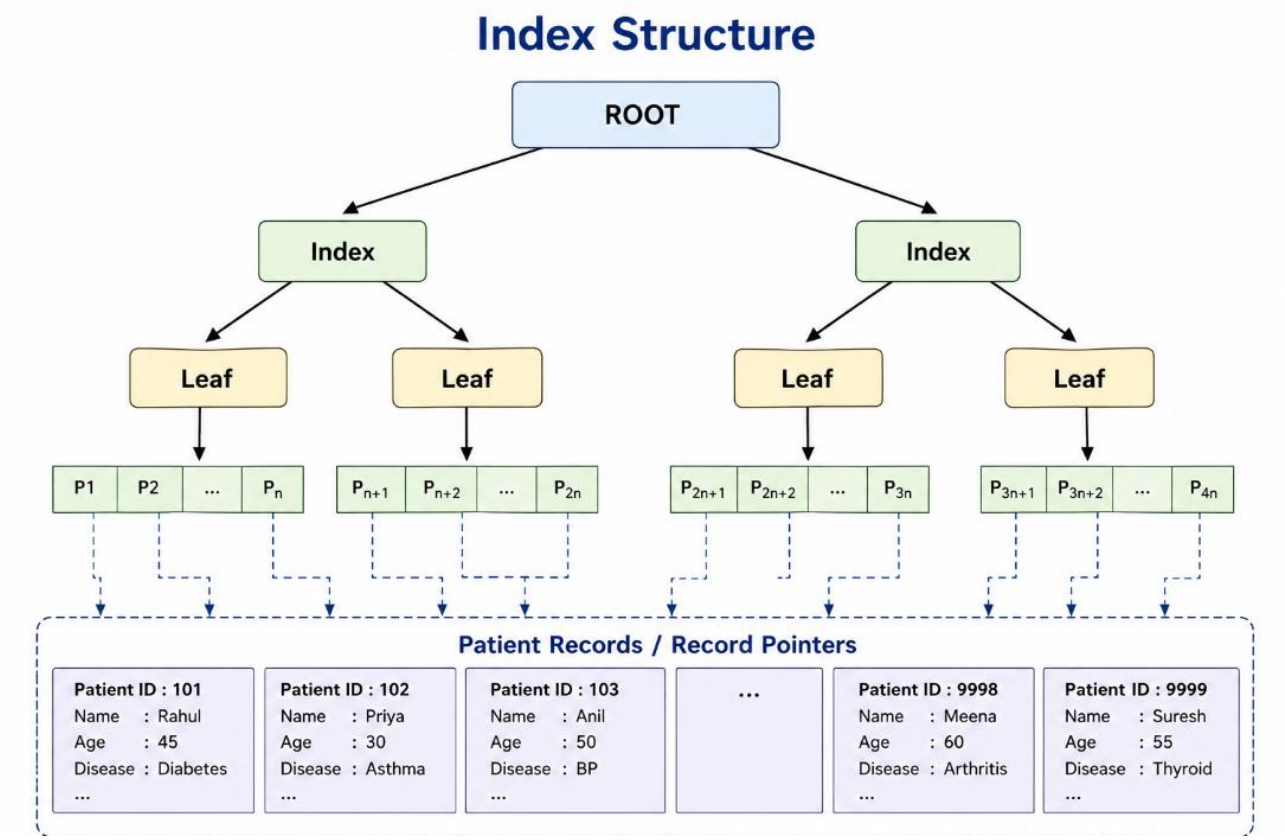
Search by PatientID:

```
SELECT *
FROM PATIENT
WHERE PatientID = 1050;
```

Search by DoctorName:

```
SELECT *
FROM PATIENT
WHERE DoctorName = 'Dr. Kumar';
```

Index Structure



Advantages

- **PatientID index** provides fast direct patient retrieval.
- **DoctorName index** provides fast retrieval of patients treated by a doctor.
- Multi-level indexing reduces the number of disk I/O operations.
- B+ Tree remains balanced even when new patient records are added.

Conclusion

A **multi-level B+ Tree index on PatientID** should be used as the primary access path, along with a **secondary B+ Tree index on DoctorName**. This supports efficient retrieval through both search requirements.

4)

B-Tree vs B+ Tree for Logistics

1. B-Tree

A B-Tree is a balanced multi-way search tree in which **keys and records can be stored in both internal and leaf nodes**.

It supports:

- Fast searching
- Dynamic insertion
- Dynamic deletion
- Balanced tree structure
- Approximately O(log n) search, insertion, and deletion

However, sequential and range access is less efficient because the leaf nodes are not directly linked.

2. B+ Tree

A B+ Tree is an improved form of B-Tree where **actual records are stored only at the leaf level**. Internal nodes contain keys and pointers for navigation.

The leaf nodes are linked:

[Shipment Records]
↓
[Leaf] → [Leaf] → [Leaf] → [Leaf]

This makes it highly suitable for large databases.

3. Comparison

Feature	B-Tree	B+ Tree
Data storage	Internal + leaf nodes	Leaf nodes only
Search	Fast	Fast
Insertion	Efficient	Efficient
Deletion	Efficient	Efficient
Range queries	Less efficient	Very efficient
Sequential access	Moderate	Very efficient
Leaf linking	No	Yes
Disk-based database	Good	Better

4. Why B+ Tree is Better for Logistics

A logistics database may frequently perform operations such as:

Find shipment by ShipmentID
Find shipments between two dates
Find shipments in a particular location
Insert new shipment
Delete completed/cancelled shipment

B+ Tree is particularly useful because its linked leaf nodes allow shipment records to be scanned sequentially without repeatedly traversing the upper levels.

For example:

Shipment Date
↓
B+ Tree
↓
[Aug 01] → [Aug 05] → [Aug 10] → [Aug 15]

The required date range can be accessed efficiently.

5. Final Recommendation

B+ Tree is more suitable for the logistics company because it provides:

- Efficient dynamic insertion and deletion
- Fast record searching
- Excellent range-query performance
- Efficient sequential access
- Better disk-based storage performance

Therefore, a **B+ Tree should be preferred over a B-Tree** for managing a large and dynamically changing shipment database.

5)

Hashing Scheme for HR System

1. Recommended Hashing Scheme

Use **EmployeeID** as the hashing key.

EmployeeID → Hash Function → Bucket

A simple hash function can be:

h(EmployeeID) = EmployeeID % 100

This provides **100 initial buckets**.

For example:

EmployeeID = 1250

1250 % 100 = 50

Therefore → Bucket 50

2. Static Hashing

In static hashing, the number of buckets is fixed when the hash file is created.

For 5,000 employees:

Number of records = 5000
Number of buckets = 100

Average records per bucket = 5000 / 100
= 50 records

Advantages

- Simple to implement.
- Fast direct access.
- Easy to manage when the number of employees is stable.

Disadvantages

- Fixed number of buckets.
- Overflow may occur when buckets become full.
- Performance decreases as the database grows.
- Reorganizing the file may be required for major growth.

3. Extendible Hashing

Extendible hashing uses a **directory and dynamically growing buckets**.

When a bucket overflows:

Bucket Overflow
↓
Bucket Split
↓
Directory Doubling (if required)

This allows the system to grow without rebuilding the entire hash file.

Advantages

- Handles dynamic employee growth.
- Reduces overflow chains.
- Efficient insertion and search.
- Directory expands only when necessary.

Disadvantage

- More complex than static hashing.
- Requires additional directory space.

4. Comparison

Feature	Static Hashing	Extendible Hashing
Buckets	Fixed	Dynamic
Overflow handling	Overflow buckets/chaining	Bucket splitting
Growth	Less flexible	Highly flexible
Implementation	Simple	More complex
Search	Fast	Fast
Suitable for changing data	Moderate	Excellent

5. Recommendation

For an HR system with **5,000 employees**, **extendible hashing is recommended** if the number of employees is expected to increase or change frequently.

If the employee count remains nearly fixed, **static hashing** is simpler and sufficient.

Conclusion
Hash Key = EmployeeID
Initial Buckets = 100

Extendible hashing is preferred for a growing HR database because it dynamically handles bucket overflow and expansion while maintaining efficient employee record retrieval.

6)

Airline Reservation System: Combined Index Structure

1. Recommended Index Structure

Use **two B+ Tree indexes**:

1. B+ Tree on FlightNo
2. B+ Tree on FlightDate

This provides efficient support for both types of queries.

Airline Table
FLIGHT

FlightNo
FlightDate
Source
Destination
Airline
AvailableSeats

2. Index on Flight Number

Create a B+ Tree index on FlightNo.

INDEX FlightNo
↓
B+ Tree
↓
Flight Record

This is useful for **point queries**, such as:

```
SELECT *
FROM FLIGHT
WHERE FlightNo = 'AI202';
```

The B+ Tree quickly locates the required flight without scanning the entire table.

3. Index on Flight Date

Create another B+ Tree index on FlightDate.

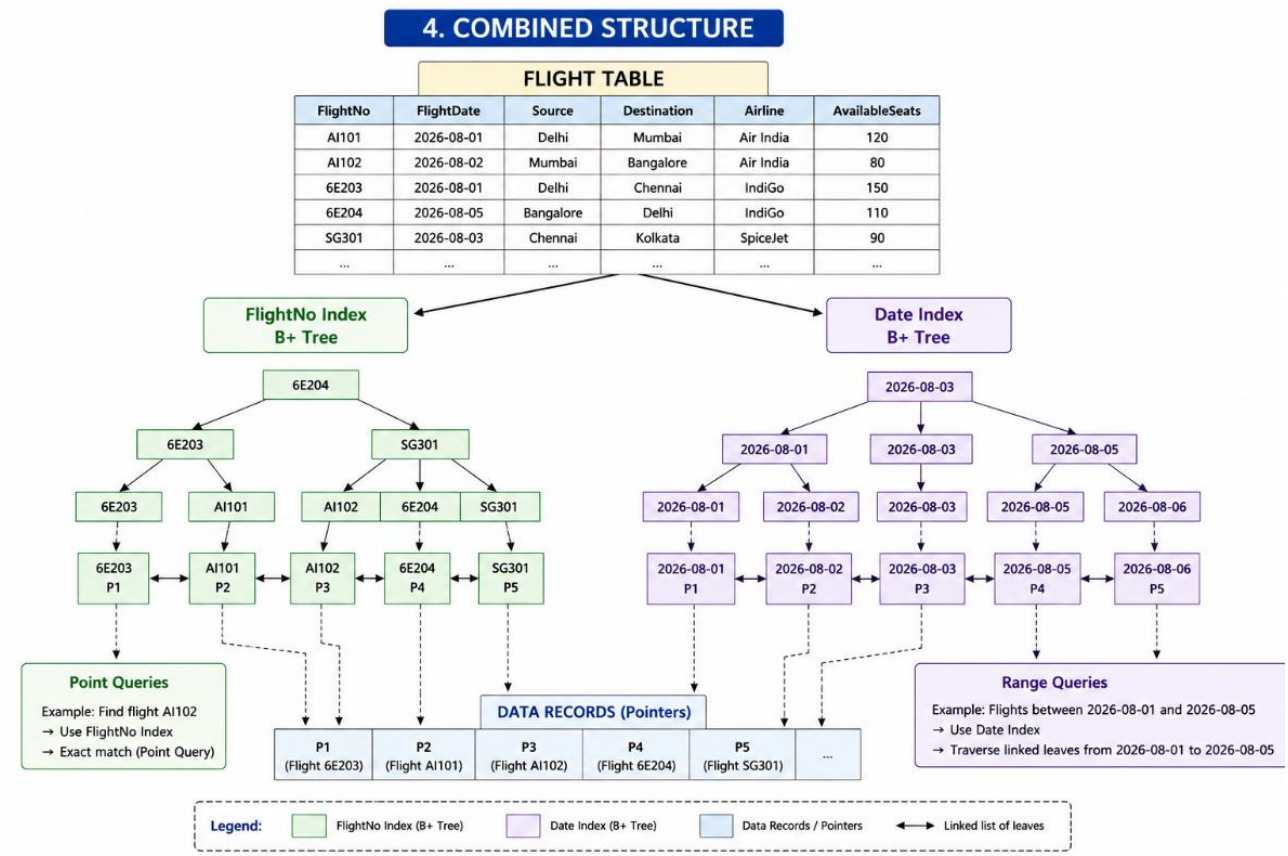
INDEX FlightDate
↓
B+ Tree
↓
Flight Records

This supports range queries such as:

```
SELECT *
FROM FLIGHT
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-15';
```

The linked leaf nodes of the B+ Tree make it efficient to retrieve all flights within the date range.

4. Combined Structure



5. Advantages

- FlightNo index provides fast **exact-match searches**.
- FlightDate index provides efficient **date-range searches**.
- B+ Trees remain balanced when flights are inserted or deleted.
- Linked leaf nodes improve sequential and range access.
- Separate indexes allow the database to choose the most suitable access path for each query.

Conclusion

The recommended solution is to use **two B+ Tree indexes: one on** FlightNo **and another on** FlightDate. The FlightNo index handles point queries, while the FlightDate index efficiently handles range queries. This provides fast retrieval and is suitable for a large airline reservation database.

7)

University Student Database

1. Recommended File Organization

Use a **sorted file organization based on Department**.

STUDENT

StudentID
StudentName
Department
CGPA
Year

Records can be grouped/sorted by department:

CSE → Students
ECE → Students
EEE → Students
MECH → Students

This makes department-based retrieval faster.

2. Secondary Index on Department

Create a **secondary B+ Tree index** on Department.

Department
↓
B+ Tree Index
↓
Student Record Pointers

Example:

CSE → Record locations
ECE → Record locations
EEE → Record locations

Since many students can belong to the same department, the index points to multiple student records.

3. Secondary Index on CGPA

Create another **B+ Tree index** on CGPA.

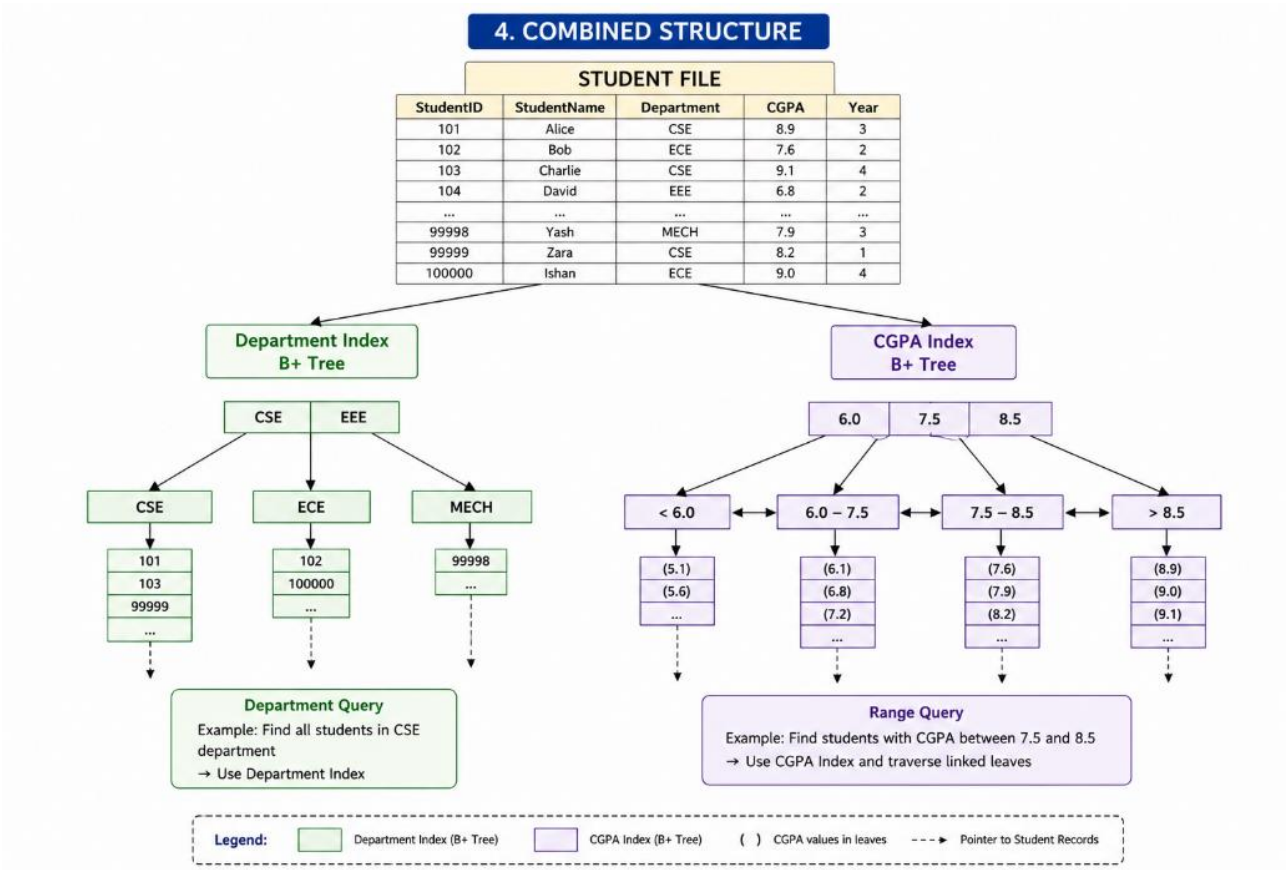
CGPA
↓
B+ Tree
↓
Student Record Pointers

This supports range queries such as:

```
SELECT *
FROM STUDENT
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

The B+ Tree can quickly locate 8.0 and then scan the linked leaf nodes until 9.0.

4. Combined Structure



5. Advantages

- Department index provides fast retrieval of students from a particular department.
- CGPA index efficiently supports **range queries**.
- B+ Tree indexes remain balanced during insertion and deletion.
- Secondary indexes avoid scanning the complete student file.
- The design is suitable for large university databases.

Conclusion

A **sorted student file with secondary B+ Tree indexes on Department and CGPA** is suitable. The Department index supports fast department-based queries, while the CGPA index provides efficient range searches such as CGPA BETWEEN 8.0 AND 9.0.

8)

Disk Block Utilization

1. Given Data
Number of records = 100,000
Record size = 50 bytes
Block size = 4 KB = 4096 bytes
2. Records per Block
Records per block = 4096 / 50
= 81.92
= 81 records

So, each block can store **81 records**.

3. Number of Blocks Required
Number of blocks = 100,000 / 81
≈ 1234.57
≈ 1235 blocks

Therefore, approximately **1235 data blocks** are required.

4. Block Utilization

Actual data stored in a full block:

81 × 50 = 4050 bytes

Unused space:

4096 - 4050 = 46 bytes

Utilization:

(4050 / 4096) × 100
≈ 98.88%

Thus, the approximate block utilization is **98.88%**.

5. Comparison

File Organization	Blocks Required	Utilization	Main Feature
Heap	≈ 1235	≈ 98.88%	Fast insertion
Sorted	≈ 1235	≈ 98.88%	Efficient ordered/range search
Hashed	≈ 1235*	≈ 98.88%*	Fast equality search

*The calculation assumes no additional overflow blocks or hash-bucket overhead.

Conclusion

All three organizations require approximately **1235 data blocks** when the records are packed similarly. The main difference is not the basic storage capacity, but **how the records are organized and accessed**:

- **Heap:** best for simple/frequent insertion.
- **Sorted:** better for ordered and range searches.
- **Hashed:** better for direct equality searches using a hash key.

9)

Composite Indexing for Social Media Posts

1. Recommended Index Structure

Use **composite B+ Tree indexes** based on the main query patterns.

POST
---
post_id
user_id
timestamp
hashtag
content

2. Index on User and Timestamp

Create a composite index:

(user_id, timestamp)

This is useful for retrieving a particular user's posts within a time range.

Example:

```
SELECT *
FROM POST
WHERE user_id = 101
AND timestamp BETWEEN '2026-08-01' AND '2026-08-15';
```

The index first locates user_id and then efficiently searches the timestamp range.

3. Index on Hashtag and Timestamp

Create another composite index:

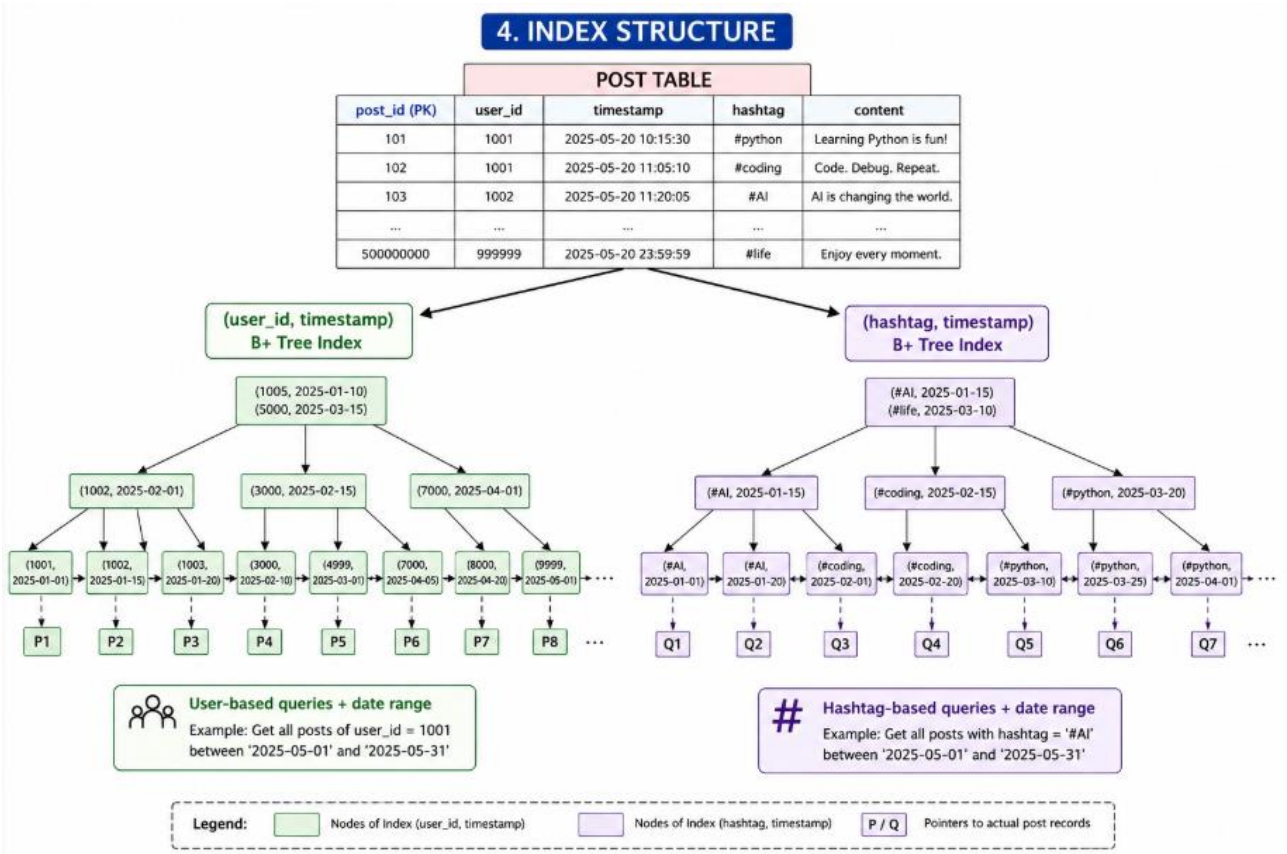
(hashtag, timestamp)

This supports queries such as finding posts under a particular hashtag during a specific period.

```
SELECT *
FROM POST
WHERE hashtag = '#technology'
```

AND timestamp BETWEEN '2026-08-01' AND '2026-08-15';

4. Index Structure



5. Why Composite Indexing?

- Supports fast queries using user_id.
- Efficiently retrieves posts within a timestamp range.
- Supports hashtag-based searches.
- Reduces the need to scan **500 million records**.
- B+ Tree indexes are suitable for both equality and range searches.

Conclusion

For a platform with **500 million posts**, the recommended strategy is to create composite B+ Tree indexes on:

(user_id, timestamp)
(hashtag, timestamp)

These indexes provide efficient user-based, hashtag-based, and time-range queries while reducing disk I/O and improving retrieval performance.

10)

Performance Comparison of File Organizations

1. Heap File

Records are stored in the order in which they are inserted.

- **Insert:** Very fast – O(1) average
- **Search:** Slow – O(n)

- **Delete:** O(n) because the record may need to be searched first
- Suitable when transactions are frequently inserted and full-file searching is acceptable.

2. Sorted File

Records are maintained in sorted order based on a key such as TransactionID.

- **Insert:** Slower because the correct position must be maintained.
- **Search:** Fast using binary search – O(log n)
- **Delete:** Slower because records may need to be reorganized.
- Suitable for ordered reports and range-based searches.

3. Hashed File

A hash function maps each transaction to a storage bucket.

Hash(TransactionID) → Bucket

- **Insert:** Fast – O(1) average
- **Search:** Very fast for exact-match queries – O(1) average
- **Delete:** O(1) average
- Collision handling is required when multiple transactions map to the same bucket.

4. Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	Very Fast	Slow	Very Fast
Search	Slow	Fast	Very Fast
Delete	Slow	Slow	Very Fast
Range Search	Slow	Very Fast	Poor
Organization	Unordered	Sorted	Hash buckets
Best Use	Frequent insertion	Ordered/range queries	Exact-match queries

5. Recommendation

For a supermarket billing system handling **1 million transactions daily**, **Hashed File Organization** is generally the best choice when transactions are mainly retrieved using an exact key such as TransactionID or BillID.

Hashing → Fast Insert + Fast Search + Fast Delete

If the system frequently generates **date-wise or amount-range reports**, a **Sorted File with a B+ Tree index** would be more suitable.

Conclusion

Heap File → Best for frequent insertion
Sorted File → Best for ordered/range queries
Hashed File → Best for exact-match billing queries

For the given high-volume supermarket billing scenario, **Hashing is recommended for primary transaction retrieval**, while an additional index can be used for reporting and range-based queries.